

课程笔记

[LLM Agents F25] 后训练可验证 Agent：从数据到算法

基于 Jiantao Jiao 授课内容整理

2026-04-02



视频作者/频道：Berkeley RDI

发布日期：2025-10-03

视频时长：1:17:28

视频链接：<https://www.youtube.com/watch?v=310Zxus34es>

项目主页：<https://github.com/hqh1025/ai-course-notes>

目录

1 课程背景与核心问题	3
1.1 本章小结	4
2 Agentic 模型与传统后训练范式的分野	4
2.1 为什么传统 RLHF 不能直接覆盖 Agent 需求	4
2.2 双目标训练: Preference + Verifiability	4
2.3 从“回答”到“行动轨迹”	4
2.4 本章小结	5
3 可验证 Agent 的任务建模: 环境、工具、状态	5
3.1 三元组视角: Environment / Tools / Verifier	5
3.2 工具调用不是附属能力, 而是主能力	6
3.3 状态转移与轨迹可验证性	6
3.4 本章小结	6
4 训练数据构建: 覆盖、组合爆炸与泛化	6
4.1 为什么数据问题在 Agent 里更难	6
4.2 数据集构建原则	7
4.3 从“看起来强”到“真实泛化”	7
4.4 本章小结	7
5 评估系统设计: 从单点评分到 Holistic Evaluation	7
5.1 Verifier 不是一个分数, 而是一组机制	7
5.2 Harness 交换与鲁棒性验证	8
5.3 评估污染与难度监控	8
5.4 本章小结	9
6 后训练主流程: Light SFT 与 RL 的分工	9
6.1 两阶段框架	9
6.2 为什么 SFT 要“轻且多样”	9
6.3 RL 阶段的目标不是“更高分”, 而是“更好学习”	10
6.4 本章小结	11
7 强化学习关键现象: 长训练、熵塌缩与探索维持	11
7.1 为什么“训练更久”不是自动有效	11
7.2 第二观察: 任务难度需要动态调度	11
7.3 第三观察: 多样高质量响应是核心资源	12
7.4 本章小结	12
8 算法细节与工程权衡: On-policy、更新强度与熵正则	12
8.1 On-policy 与 Off-policy 的核心差异	12
8.2 更新强度控制: clipping 与解耦阈值	13
8.3 熵正则: 显式维持可探索空间	13

8.4 本章小结	14
9 研究前沿与开放问题	14
9.1 理论与实践之间仍有明显断层	14
9.2 从人类学习类比到可实现算法	14
9.3 部署风险与治理议题	14
9.4 本章小结	14
10 附录：工程落地检查清单	15
10.1 训练前准备清单	15
10.2 训练中监控面板	16
10.3 训练后验收与发布门槛	16
10.4 本章小结	16
11 附录：讲座时间线精读	16
11.1 关键时间点与技术主线	16
11.2 关键术语对照	18
11.3 本章小结	18
12 总结与延伸	19
12.1 全讲框架回顾	19
12.2 延伸阅读与实践建议	19
12.3 进一步思考	20

1 课程背景与核心问题

本讲主题是 **Post-Training Verifiable Agents**。Jiantao Jiao 的切入角度非常明确：如果我们希望 Agent 在真实环境里长期稳定执行任务，仅靠“对话质量好”不够，必须把训练目标从“讨好人类偏好”扩展为“在环境反馈下可验证地完成任务”。也就是说，模型输出不再只是文本，而是带状态影响的行动轨迹 (trajectory)。

本讲主命题

Agentic 模型的后训练目标可以写成一句话：**在保持人类可用性的前提下，最大化可验证任务回报 (verifiable rewards)**。它要求系统同时满足“会沟通”与“会做事”两类能力，而不是只优化其中一侧。

讲者在开场就强调，很多早期聊天模型主要优化的是人类偏好 (preference alignment)，例如回答礼貌、语气自然、内容看起来合理。但 Agent 任务往往包含代码修改、工具调用、数据库查询、外部 API 交互等动作，这些动作会改变环境状态，错误也会有累计效应。因此，训练目标需要显式纳入环境反馈。

从 Chatbot 到 Agent 的目标函数变化

- Chatbot：偏向“回答是否让人满意”，奖励常来自偏好比较。
- Agent：偏向“任务是否完成且可验证”，奖励更多来自程序化 verifier。
- 两者并非对立：生产系统通常需要“偏好质量 + 任务正确率”的联合优化。

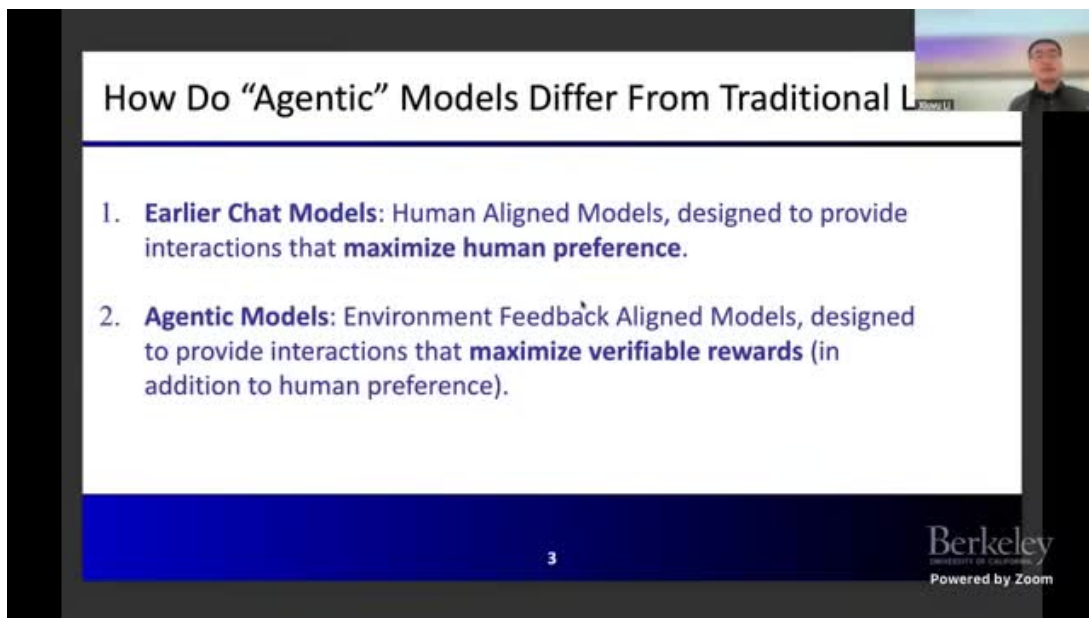


图 1: 讲座前段对 Agentic shift 的定义：从聊天能力转向环境对齐

课程还反复指出一个实践问题：当模型开始执行复杂任务时，“看起来像对”与“真的做对”差距会迅速扩大。越是多步任务，这个差距越明显。因此，是否有高质量 verifier，以及 verifier 是否

⁰关键帧来源：原视频，时间约 00:01:34。

覆盖关键失败模式，直接决定后训练上限。

1.1 本章小结

本章建立了后续全部讨论的前提：Agent 后训练不是把聊天模型“再微调一下”，而是把训练目标重构为“环境中的可验证任务完成”。这会牵引数据、评估和算法三条线同时变化。

2 Agentic 模型与传统后训练范式的分野

2.1 为什么传统 RLHF 不能直接覆盖 Agent 需求

传统 RLHF 的核心是偏好建模：收集人类比较数据，训练 reward model，再用 SFT、PPO/GRPO 等算法提升“被人偏好”的概率。这条链路在对话系统上很有效，但在 Agent 场景里会暴露两个缺口：第一，许多任务存在硬约束（例如 JSON 结构、单元测试通过、SQL 正确执行）；第二，任务执行过程会跨越多轮工具调用，最终结果取决于过程正确性而非表面表达。

常见误区：把 Agent 训练当成“更长的对话 RLHF”

如果只把 Agent 当作更长上下文的聊天任务，会出现三类问题：

- 过程错误被最终话术掩盖，导致“可读不可用”；
- 奖励信号过度依赖主观偏好，难以覆盖硬性正确性；
- 训练后模型在真实工具链中不稳定，出现格式漂移和动作失配。

2.2 双目标训练：Preference + Verifiability

讲者的建议不是抛弃偏好目标，而是建立双目标后训练框架：模型既要保留自然交互能力，又要在环境中完成可验证任务。具体做法通常是让数据集和评估集同时包含“偏好维度”与“任务维度”，并在训练阶段分配不同权重。

维度	传统对话后训练	可验证 Agent 后训练
训练对象	单轮或短多轮文本输出	长轨迹动作序列（含工具调用）
奖励来源	人类偏好模型为主	verifier + 偏好联合
失败表现	不够礼貌或不够有用	任务失败、格式失败、状态污染
验证成本	高度人工主观判断	可程序化自动验证比例更高

表 1: 两种后训练范式的核心差异

2.3 从“回答”到“行动轨迹”

课程中的一个关键表述是：Agent 的核心单位不是 token，而是 trajectory。一次任务里模型需要先理解目标，再决定何时调工具、如何解释工具反馈、是否需要重试、何时终止并汇报。这个过程本质是序列决策问题。

后训练单位变化

当训练单位从“response”变为“trajectory”后，数据标注、奖励计算、评估指标、甚至日志系统都要改变。很多工程团队的瓶颈并不在模型本身，而在没有把这套基础设施同步升级。

2.4 本章小结

本章结论是：Agent 后训练必须从“偏好对齐”升级为“偏好 + 可验证任务完成”的联合优化，并把训练单位重定义为可执行轨迹，而非单条文本回复。

3 可验证 Agent 的任务建模：环境、工具、状态

3.1 三元组视角：Environment / Tools / Verifier

在讲座中，讲者将可验证 Agent 数据与训练对象拆成三个核心部分：环境（environment）、工具（tools）、验证器（verifier）。这不是概念分类，而是可执行系统中的三类实际约束。

Environment 的含义

Environment 不只是“题目文本”，而是包含运行上下文的完整状态：代码仓库文件、依赖版本、测试脚本、系统提示词、可访问接口、历史交互痕迹等。Agent 的每个动作都可能改变该状态。

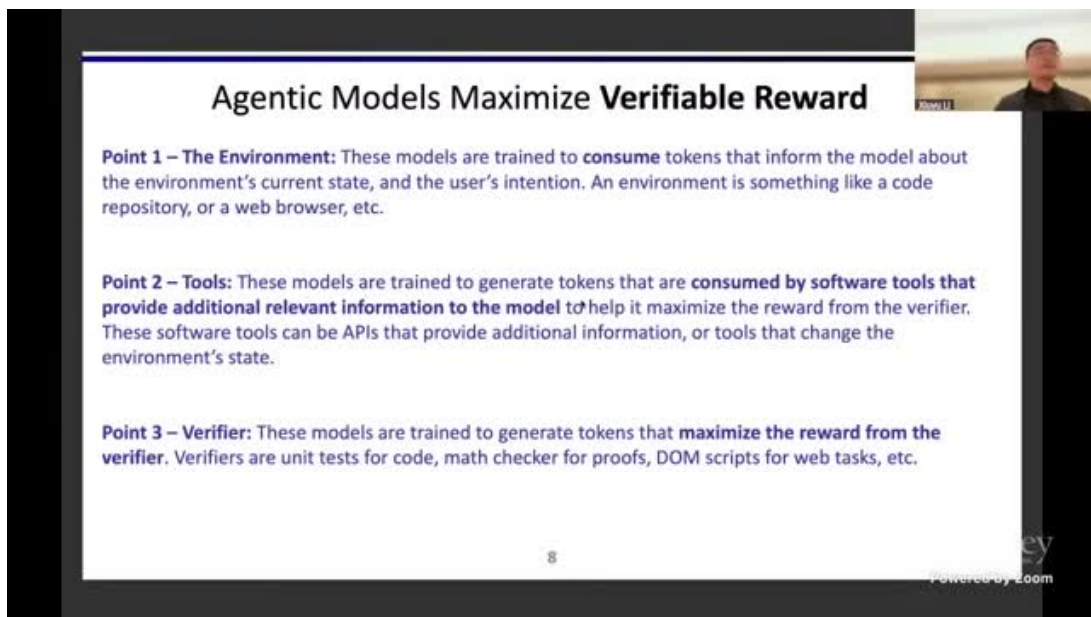


图 2: 环境状态建模示意：任务目标、代码仓库、系统配置与上下文共同构成训练环境

⁰ 关键帧来源：原视频，时间约 00:12:30。

3.2 工具调用不是附属能力，而是主能力

讲者强调，Agentic LLM 的一个核心能力是生成可被工具消费的 token 序列。也就是说，模型不仅要“会说”，还要“会调用”：参数正确、调用时机正确、错误恢复路径正确。工具链越复杂，训练数据越需要覆盖动作空间差异。

工具调用的三层正确性

- 语法正确：格式、字段、类型、JSON 结构合法。
- 语义正确：调用意图匹配任务目标，参数有业务意义。
- 时序正确：在正确阶段调用正确工具，避免无效循环。

3.3 状态转移与轨迹可验证性

Agent 在环境中执行任务时，关键不只是最终答案，而是状态转移是否可追踪。例如代码修复任务中，“改了什么”、“为何改”、“测试是否覆盖”都应被记录并可验证。这就要求训练样本包含完整轨迹，而不只是终局结果。

只监督终局答案的风险

如果数据只给“最终通过”样本，模型可能学到不可泛化的捷径：

- 对中间步骤缺乏约束，导致不可解释行为；
- 面对分布外任务时，缺乏稳定的中间决策策略；
- 调试困难，错误定位成本飙升。

3.4 本章小结

可验证 Agent 的建模核心是把问题写成“环境状态 + 工具动作 + 验证反馈”的闭环系统。训练质量取决于这三者是否被同时建模，而非只强化最终文本回答。

4 训练数据构建：覆盖、组合爆炸与泛化

4.1 为什么数据问题在 Agent 里更难

讲者把“高质量训练数据”放在第一优先级。原因在于 Agent 任务空间是组合爆炸的：环境类型、工具集合、验证方式、任务目标可自由组合。即便在单一领域（如 coding），也会出现仓库结构、测试风格、错误类型、依赖生态的大幅变化。

Agent 数据难点是组合，而非规模

在 Agent 训练中，单纯增加样本数量并不能保证泛化。真正关键是是否覆盖了“环境-工具-验证器”的代表性组合，以及这些组合之间是否有足够多的跨域迁移信号。

4.2 数据集构建原则

结合讲座内容，可以将可验证 Agent 的数据构建原则归纳为以下四点：任务类型广度、动作多样性、验证维度完整性、失败模式显式化。尤其是失败样本，往往比成功样本更能提升鲁棒性。

原则	目标	落地做法
环境多样性	减少场景过拟合	同时引入数学、代码、检索、业务 API 等环境
动作多样性	提升策略广度	对同一任务保留多条高质量轨迹而非单解
验证多样性	防止单指标投机	单元测试、格式检查、规则引擎、人工抽检联合
失败样本显式化	学习错误边界	记录失败原因并在训练中加入负反馈

表 2: 可验证 Agent 数据构建的四条主线

4.3 从“看起来强”到“真实泛化”

讲者反复提醒：如果训练集与评估集过于相似，模型会出现“指标高但真实任务崩”。这在 Agent 场景更严重，因为任务链路更长，任何一步分布偏差都会放大终局误差。

数据-评估同分布幻觉

若训练和评估过于接近，会导致团队误判模型能力：

- 在已见任务上持续提升，但在新任务上性能急剧下降；
- 模型学会针对 benchmark 的策略，而非学到一般能力；
- 部署后出现“离线好、在线差”的系统性偏差。

4.4 本章小结

Agent 数据工程的重点是“覆盖正确的组合空间”。与其追求样本总量，不如优先保障环境、工具、验证器和失败模式的结构化多样性。

5 评估系统设计：从单点评分到 Holistic Evaluation

5.1 Verifier 不是一个分数，而是一组机制

课程中一个很重要的观点是：verifier 不应被理解为单个布尔规则，而应是多维度验证体系。代码任务可能用单元测试，数学任务可用 proof checker，结构化输出任务要校验 JSON 合法性与字段一致性，业务任务还要检查副作用边界。

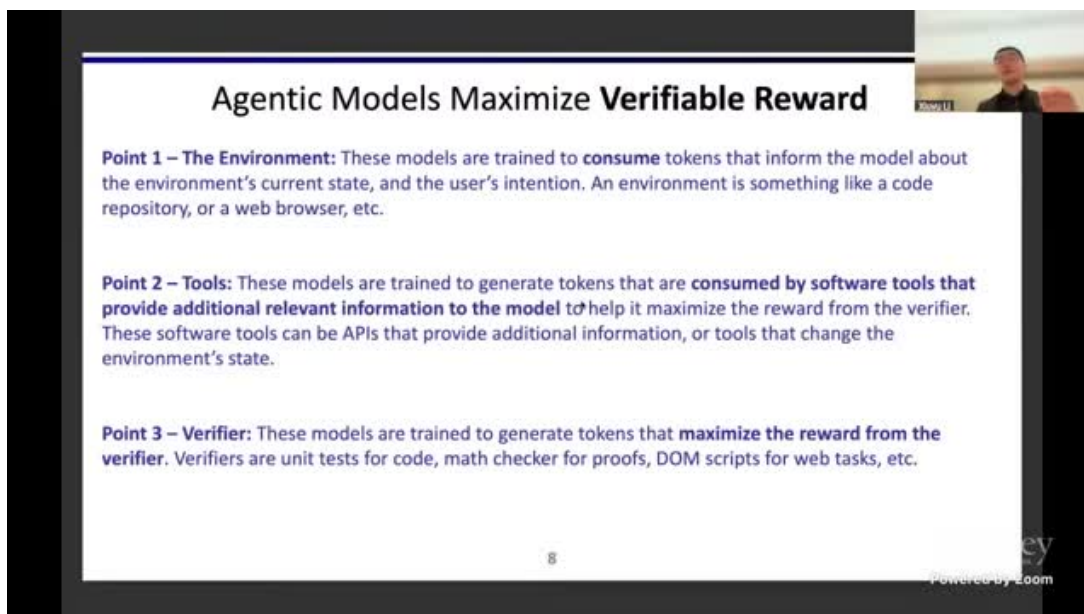


图 3: 多视角 verifier 设计：正确性、格式、过程约束联合检查

Verifier 的分层设计

- L1 结果层：最终答案是否正确。
- L2 结构层：输出是否满足协议与格式要求。
- L3 过程层：关键中间步骤是否符合策略约束。
- L4 安全层：是否触发越权调用、数据泄露或危险动作。

5.2 Harness 交换与鲁棒性验证

讲者提到应在评估中引入 harness swapping：同一任务在不同执行框架、不同工具包装、不同系统提示下重复评测。如果性能大幅波动，往往说明模型依赖了偶然实现细节而非稳健策略。

Harness 交换的价值

它不是“再测一次”，而是检测模型是否具备环境迁移鲁棒性。若模型只在某个固定 harness 上表现优秀，部署风险会很高。

5.3 评估污染与难度监控

课程中还强调 benchmark contamination 与 hardness drift。随着社区迭代加速，公开数据可能被训练过程吸收，导致指标失真。评估系统必须持续监控题目难度与分布变化。

⁰关键帧来源：原视频，时间约 00:15:08。

高分不等于高能力

当 benchmark 被污染或难度下降时，高分可能只意味着“见过类似题”。因此评估要包含：

- 新样本引入频率控制；
- 题目泄露风险审计；
- 难度分层报告 (easy/medium/hard)；
- 跨 benchmark 交叉验证。

5.4 本章小结

可验证 Agent 的评估必须是 Holistic 的：多 verifier、多 harness、多难度分层、持续污染监控。单一分数无法支撑可靠部署。

6 后训练主流程：Light SFT 与 RL 的分工

6.1 两阶段框架

讲者给出的实践框架是两阶段：先做 **Light SFT**，再做 **RL 强化**。SFT 的作用是把模型带入“可用轨道”，减少无意义动作；RL 的作用是让模型在反馈中持续探索并提升真实任务能力。

分工边界

- SFT：建立初始行为先验，压低明显错误率。
- RL：在可验证反馈下优化策略，提升复杂任务成功率。

如果 SFT 过重，模型会过度模仿演示、探索能力下降；如果 SFT 过轻，RL 早期会浪费大量算力在低质量轨迹上。

6.2 为什么 SFT 要“轻且多样”

讲座明确提出 two requirements：SFT 样本要多样，且训练强度要轻。原因是 Agent 需要在 RL 阶段保持探索空间。过重 SFT 会把分布压得过窄，后续即便加大 RL 计算，也难以跳出早期模式。

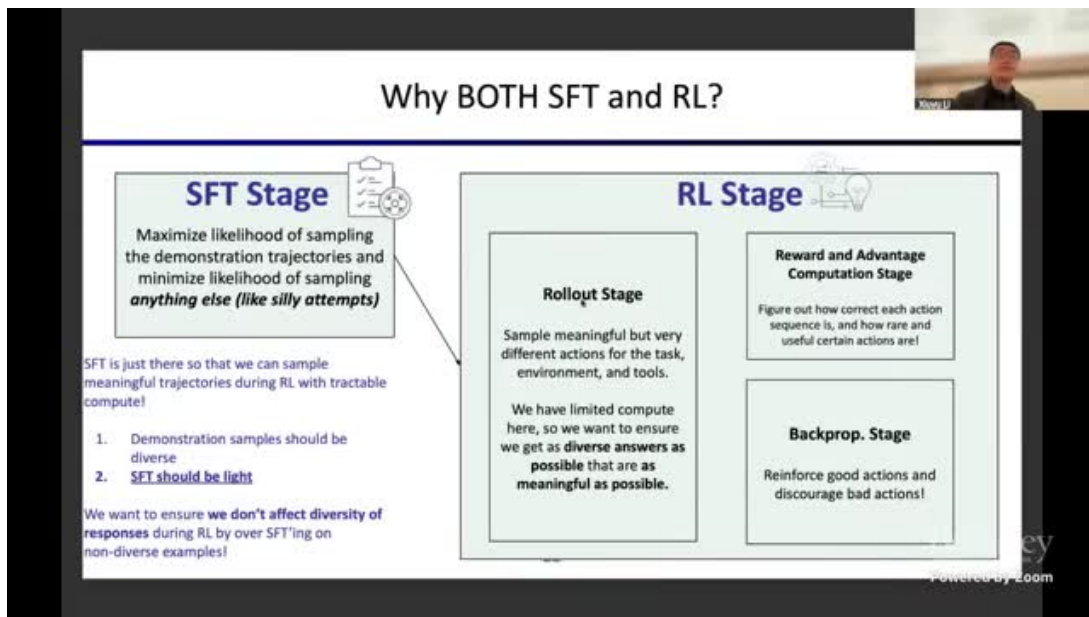


图 4: SFT 与 RL 的衔接关系：先保证可行轨迹，再强化多样探索

轻 SFT 的工程判据

实践中可用如下信号判断 SFT 是否过重：

- 训练后采样熵快速下降；
- 同任务轨迹高度同质化；
- RL 初期奖励提升快但很快停滞；
- OOD 任务表现明显劣化。

6.3 RL 阶段的目标不是“更高分”，而是“更好学习”

讲者指出，RL 的关键在于让模型从自身错误里学习，而不仅是追求短期 reward 增长。换言之，反馈机制要能区分“可修复错误”与“策略性错误”，并把更新集中在可提升区域。否则会出现训练奖励上涨、测试泛化不升反降。

只看训练 reward 的风险

如果团队只盯训练 reward 曲线，容易忽略：

- 探索空间塌缩 (entropy collapse)；
- 对易题过拟合、对难题无增益；
- 在验证器缺陷处投机。

⁰关键帧来源：原视频，时间约 00:46:25。

6.4 本章小结

本章的核心是两句话：**SFT 负责起步，不负责封顶；RL 负责突破，但前提是保持可探索性。**
Light SFT + 强反馈 RL 是当前可验证 Agent 的主流工程路径。

7 强化学习关键现象：长训练、熵塌缩与探索维持

7.1 为什么“训练更久”不是自动有效

讲者提出第一个观察：很多系统在 RL 中会很快进入性能平台期。根因之一是输出熵下降过快，模型只会重复高概率轨迹，难以继续发现新解。对长任务 Agent 来说，这意味着“能做一点，但做不深”。

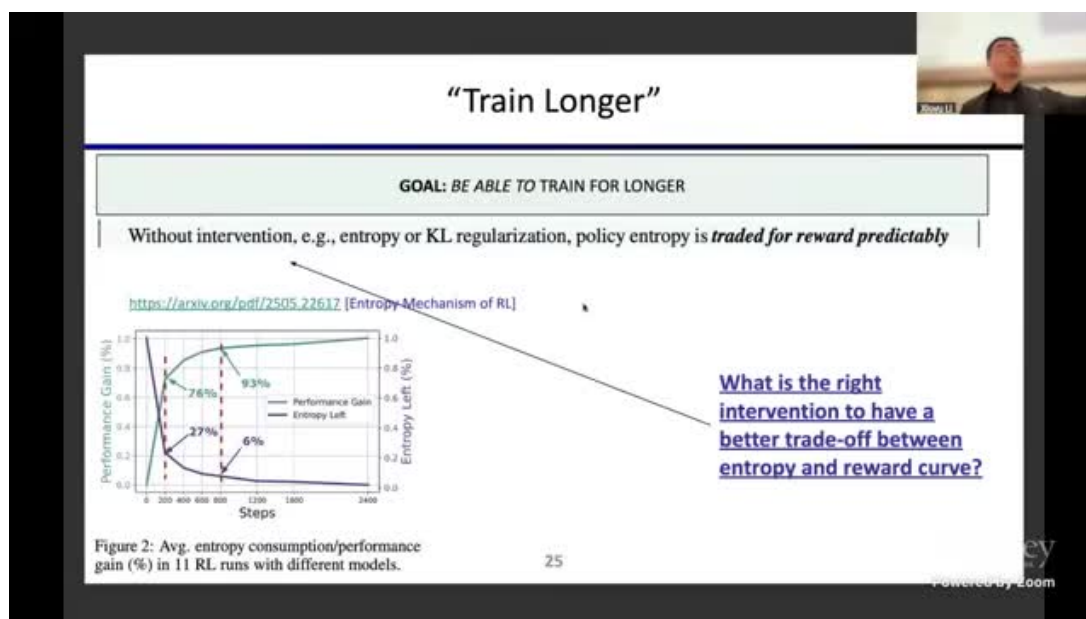


图 5: 熵塌缩现象：reward 可上升，但探索多样性下降导致长期停滞

熵塌缩的实质

熵塌缩不是“模型变差”，而是“模型变窄”：它在少数轨迹上越来越确定，但失去探索不同正确路径的能力。对于复杂任务，这会直接限制上限。

7.2 第二观察：任务难度需要动态调度

如果训练样本长期停留在简单任务，模型很快“学完”，后续更新收益极低；但如果一开始就喂过难任务，也会因为正反馈稀疏而学不动。因此，课程隐含了一个 curriculum 观点：难度应随能力上升动态调整。

⁰关键帧来源：原视频，时间约 00:58:30。

难度调度的实践框架

- 起步阶段：高可验证、低复杂度任务保证有效学习信号。
- 中期阶段：提高组合复杂度，增加多工具协作场景。
- 后期阶段：引入长轨迹、弱监督、开放式问题，重点保探索。

7.3 第三观察：多样高质量响应是核心资源

课程把“diverse high-quality responses”作为第三支柱。这里的多样不是随机噪声，而是多条可行策略路径。只有当正负样本都具有结构多样性，RL 才能学到“什么时候该怎么做”，而不是死记模板。

伪多样性的陷阱

如果多样性只体现在表面措辞，而动作序列和决策逻辑仍然单一，训练效果会被高估。真正有效的多样性应体现在工具选择、步骤顺序、失败恢复策略等决策层。

7.4 本章小结

长训练要有效，必须同时处理三件事：防熵塌缩、做难度调度、保证策略级多样性。否则训练越久，收益越小。

8 算法细节与工程权衡：On-policy、更新强度与熵正则

8.1 On-policy 与 Off-policy 的核心差异

讲者用直观方式解释了 on-policy：模型应优先从“自己当前策略生成的轨迹”中获得反馈。因为这些错误最贴近当前能力边界，学习效率更高。对 Agent 训练来说，这通常能更好地维持策略一致性和探索质量。

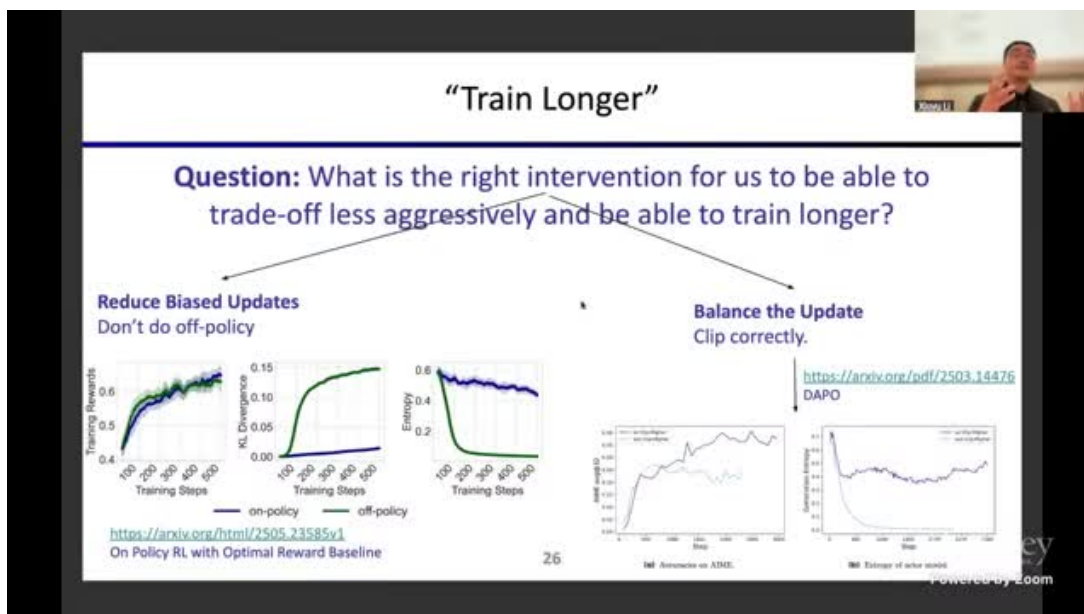


图 6: On-policy 与 Off-policy 对比：奖励增长、熵变化与稳定性的平衡

工程上如何理解 on-policy 优势

on-policy 的价值不是理论口号，而是减少“反馈错位”：你给模型的奖励，最好对应模型当前真实会走的轨迹，而不是别的策略分布。这样更新方向更一致，避免学到不可复现行为。

8.2 更新强度控制：clipping 与解耦阈值

课程讨论了 RL 更新中的 clipping 调整：通过上/下界非对称设置，鼓励低概率 token 的合理探索，缓解过早收敛。讲者也坦诚这类技巧在当下更像“有效工程 hack”，未必是长期最优理论形式。

工程结论

在可验证 Agent 训练里，**稳定更新**与**保持探索**同等重要。只追求稳定会导致保守退化，只追求探索会导致训练发散。clipping、熵正则等机制本质上是在做这两个目标的平衡。

8.3 熵正则：显式维持可探索空间

当观测到熵持续下降时，常见做法是加入熵相关损失项，将生成熵维持在合理区间。讲座中提到社区已有多种实现方案，效果依赖任务、模型与 verifier 质量，尚不存在一次性通吃的方法。

不要把技巧当定律

当前许多有效技巧是“经验可行”，并不代表理论已收敛。团队在迁移方法时应保留审计和消融流程，避免把某个数据集上的增益误判为普适规律。

⁰关键帧来源：原视频，时间约 01:00:50。

8.4 本章小结

算法层面最关键的不是某个单一公式，而是“反馈一致性 + 更新稳定性 + 探索维持”三者协同。on-policy、clipping 调整、熵正则都是围绕这三点展开。

9 研究前沿与开放问题

9.1 理论与实践之间仍有明显断层

讲者明确指出，当前主流后训练算法与理想化机器学习理论并未完全对齐。实践中很多成功经验来自规模化试验和工程迭代，而不是闭式推导。因此，未来几年算法空间仍非常开放。

开放研究方向

- 更贴合 Agent 场景的策略优化目标；
- 更稳健的长轨迹 credit assignment；
- 更低成本的高质量 verifier 构建；
- 训练与部署一体化的在线反馈闭环。

9.2 从人类学习类比到可实现算法

课程多次使用“人类学习”做类比：先看示例，再做题，再拿反馈迭代。类比本身直观，但真正困难在于把它转成可训练、可扩展、可审计的算法与系统。这一点在 Agent 场景尤为突出。

类比可用，直译不可用

把人类学习过程直接翻译成算法通常会失败。可行路径是提取可操作原则，例如：

- 先建立基本可行策略（Light SFT）；
- 再以高质量反馈持续探索（RL）；
- 始终监控泛化与评估污染（Holistic eval）。

9.3 部署风险与治理议题

当 Agent 被用于代码、业务流程或关键系统，训练问题会直接转化为治理问题。模型是否会越权调用、是否会在弱验证场景投机、是否能被追责审计，都需要在训练和评估阶段前置设计。

能力提升必须伴随可控性提升

如果只提升任务成功率而不提升可解释性、可回滚性和权限治理，系统风险会随能力线性以上增长。可验证训练的真正价值，正在于把能力增长与治理能力绑定在一起。

9.4 本章小结

本章结论是：可验证 Agent 仍处于快速演化期，方法论尚未收敛。最值得投入的方向是把“高质量反馈学习”与“可部署治理机制”联合起来。

10 附录：工程落地检查清单

10.1 训练前准备清单

为了把课程中的方法真正落地，团队通常需要先完成“训练前系统准备”。这一环节经常被低估，但它决定了后续 RL 是否能稳定推进。尤其是可验证 Agent 训练，任何日志缺失或工具观测缺失都会直接削弱反馈质量。

优先级排序建议

先把“可观测”做完整，再追求“算法更强”。如果缺少高质量轨迹日志、工具调用回放和 verifier 诊断信息，再复杂的算法也会因为反馈噪声而收益有限。

阶段	检查项	通过标准
任务定义	任务目标可程序化表达	对每类任务可写出明确 success/failure 条件
任务定义	终止条件明确	可判定“何时应停止继续调用工具”
环境建模	环境状态可序列化	能回放每一步状态变更并定位差异
环境建模	环境重置机制可靠	同一任务可重复运行并得到可比较结果
工具链路	工具 I/O 协议固定	字段、类型、错误码、超时策略均有文档
工具链路	工具副作用可审计	任意调用可追溯到操作者、参数、时间与结果
Verifier	结果层 verifier 完整	关键任务均有自动通过/失败判断
Verifier	过程层 verifier 可用	对关键中间步骤可检测常见违规模式
数据管线	轨迹采集字段齐全	Prompt、action、observation、reward 全量记录
数据管线	失败样本可分型	至少区分工具失败、策略失败、验证器失败
训练系统	采样吞吐可监控	可实时观察样本速率、成功率、拒绝率
训练系统	更新稳定性可监控	可追踪梯度异常、loss 爆炸、熵突降等事件
评估系统	难度分层报告可产出	easy/medium/hard 分层结果可自动生成
评估系统	Harness 交换机制可运行	同任务可在不同 harness 下重复评测

阶段	检查项	通过标准
治理与安全	权限边界已实现	高风险工具调用需满足显式授权策略
治理与安全	回滚链路可演练	关键失败可在分钟级回滚到安全状态

10.2 训练中监控面板

结合课程中的观察，团队在训练过程中至少应维护三块看板：能力看板（准确率/成功率）、探索看板（熵/轨迹去重率）、可靠性看板（格式正确率/工具错误率）。三者缺一不可。

推荐日常监控指标

- 任务成功率：按任务类别和难度分层统计，不看单一均值。
- 输出熵与去重率：监控探索是否过早塌缩。
- verifier disagreement：不同 verifier 对同轨迹是否冲突。
- 工具失败率：按工具类型和错误码分布聚类。
- 轨迹长度分布：防止出现无意义超长循环。

10.3 训练后验收与发布门槛

课程强调“高分不等于可部署”。因此，训练后验收不能只看 benchmark 分数，还要看跨环境稳定性、异常场景恢复能力、权限边界遵守率。发布门槛必须写成制度化 checklist，而非临时判断。

发布前必须回答的三个问题

- 这个模型在分布外任务失败时，是否会安全失败（safe fail）？
- 当 verifier 冲突或不可用时，系统是否会降级而非盲目执行？
- 线上事故发生后，是否能在审计日志中还原完整责任链？

10.4 本章小结

工程落地的关键不是“再加一种算法”，而是把任务定义、状态观测、验证体系、监控面板和发布门槛连接成闭环。只有闭环完整，课程中的方法才会在真实系统里持续生效。

11 附录：讲座时间线精读

11.1 关键时间点与技术主线

为了便于复习，本节按照时间线提炼讲座主线，并对应到可执行工程动作。此表既可用于复盘，也可直接转为团队内部的学习任务分配。

时间	讲座要点	工程启示
00:00–00:03	主 题 定 义: Post-Training Verifiable Agents	明确目标不是“更会聊”,而是“更会完成可验证任务”
00:03–00:06	回 顾 RLHF/SFT/P-PO/GRPO 基 本 链 路	将原有聊天后训练管线扩展到轨迹级学习
00:06–00:10	可 验 证 任 务 需 要 清 晰 verifier	建立自动验证优先的数据与评估标准
00:10–00:14	coding agent 环境 与 工 具 状 态 建 模	数据采集要记录状态转移,不只记录最终答案
00:14–00:18	verifier 多样性与覆盖挑 战	使用多维 verifier 组合替代单评分器
00:18–00:22	训练与评估分布差异风 险	评估集要做去污染和难度漂移监控
00:22–00:26	泛化不是由单 bench- mark 决定	建立跨任务、跨工具、跨 harness 的评估矩阵
00:26–00:31	structured output 与 真 实 可 用 性 问 题	把格式正确率纳入硬门槛,不靠后处理兜底
00:31–00:34	holistic evaluation 与 benchmark contamination	引入持续更新评估池与社区协作审计机制
00:34–00:40	进入训练策略讨论前的 前提重申	先保证数据与评估质量,再谈算法优劣
00:40–00:44	两阶段框架: SFT + RL	Light SFT 打底, RL 负责探索与提升
00:44–00:49	轻 SFT + 多样样本的 重要性	避免过重 SFT 把策略空间压窄
00:49–00:54	算法仍在演化,理论未 收敛	保持消融实验与多路线并行探索
00:54–00:58	train longer / harder tasks / diversity 三观察	训练策略需同时考虑步数、难度和多样性
00:58–01:00	entropy collapse 现象分 析	建立熵监控与探索保持机制,防止早停滞
01:00–01:03	on-policy 与 off-policy 对比	优先从当前策略轨迹学习,减少反馈错位
01:03–01:05	clipping 解耦与探索鼓 励技巧	在稳定与探索之间做可解释的参数平衡
01:05–01:08	熵正则等方法缓解塌缩	可显式加入熵控制损失,结合任务表现调参
01:08–01:14	难题训练集构造与能力 分层观察	建立按难度分桶的数据调度机制

时间	讲座要点	工程启示
01:14– 01:17	Q&A 与系统化建议收 束	将课程结论转化为团队流程、面板 与发布门槛

11.2 关键术语对照

术语	本讲语境	建议团队内定义
Verifiable reward	可程序化判定的任务反馈	可自动复现、可审计、可分解的奖励信号
Environment state	任务运行上下文全状态	含输入、文件、配置、历史动作与观测
Trajectory	多步行动与反馈序列	训练、评估与回放的统一数据主键
Light SFT	轻量行为先验注入	减错而不压缩探索, 保持 RL 可提升空间
Entropy collapse	采样多样性丢失	策略退化为单一路径, 导致长期停滞
Harness swapping	执行框架扰动评测	测试模型是否依赖偶然实现细节

表 5: 课程关键词的工程化对照

如何使用本附录

建议将时间线和术语表直接映射到团队周计划：每个时间段对应一个工程行动项，每个术语对应一个内部统一定义，避免跨角色沟通歧义。

11.3 本章小结

时间线精读的价值在于把“听懂”变成“可执行”。当每个关键观点都对应到具体工程动作，课程内容才能真正进入团队生产体系。

12 总结与延伸

12.1 全讲框架回顾

模块	核心结论	实践动作
问题定义	Agent 后训练目标是可验证任务完成	建立 preference + verifier 联合目标
任务建模	核心三元组：环境、工具、验证器	数据与系统日志按轨迹组织
数据工程	难点在组合覆盖而非样本总量	扩展环境/工具/验证组合，显式纳入失败样本
评估体系	必须 Holistic，防止指标幻觉	多 verifier + harness swapping + 污染监控
训练流程	Light SFT 打底，RL 负责突破	控制 SFT 强度，保留探索空间
RL 关键点	防熵塌缩、难度调度、多样高质轨迹	监控熵与难度分层，优化采样与反馈闭环
算法实现	on-policy、更新强度平衡、熵正则有用	保留消融与审计，避免技巧神化

表 6: Post-Training Verifiable Agents 的工程化总图

一句话总结

可验证 Agent 训练的本质，不是“让模型更会说”，而是“让模型在可观测、可验证、可治理的环境中持续学会做对事”。

12.2 延伸阅读与实践建议

主题	建议阅读/实践方向
课程原视频	Berkeley RDI 发布的本讲完整视频 (Jiantao Jiao, Post-Training Verifiable Agents)，建议结合本笔记的关键时间戳回看图示部分。
评估工程	重点补齐 verifier 分层设计：结果层、结构层、过程层、安全层；在内部基准上进行 harness swapping 实验。
训练策略	对比“重 SFT + 轻 RL”与“轻 SFT + 强 RL”两种配置，追踪熵变化、成功率和 OOD 泛化差异。
熵与探索	针对熵塌缩建立日常监控面板：token entropy、轨迹去重率、有效探索比例、失败类型分布。
数据体系	从“按任务存样本”升级到“按轨迹存状态转移”，将工具调用参数与中间反馈纳入训练可回放数据。

主题	建议阅读/实践方向
治理闭环	引入权限边界、调用审计、异常回滚机制，把部署可控性作为后训练优化目标的一部分。

12.3 进一步思考

- 当 verifier 无法覆盖复杂开放任务时，如何避免模型在弱约束区域形成投机策略？
- 在成本受限条件下，如何选择最具信息增益的轨迹进行 on-policy 更新？
- 可验证 Agent 的评估是否应从“单模型分数”转向“系统级可靠性曲线”？
- 如果训练目标同时包含能力与治理，损失函数和评估基准应如何共同设计？